ODAT-SE SIM-TRHEPD-RHEED Module Documentation

Release 1.1.0

ISSP, University of Tokyo

Oct 10, 2026

CONTENTS:

1 Introduction **1**
- 1.1 What is ODAT-SE ? . . . 1
- 1.2 What is odatse-STR ? . . . 1
- 1.3 License . . . 1
- 1.4 Version Information . . . 2
- 1.5 Main developers . . . 2

2 Installation of odatse-STR **5**
- 2.1 Prerequisites . . . 5
- 2.2 How to download and install . . . 5
- 2.3 How to run . . . 6
- 2.4 How to uninstall . . . 6

3 Tutorials **7**
- 3.1 Direct Problem Solver . . . 7
- 3.2 Optimization by Nelder-Mead method . . . 12
- 3.3 Optimization by Grid search . . . 16
- 3.4 Optimization by Bayesian Optimization . . . 21
- 3.5 Optimization by replica exchange Monte Carlo method . . . 27
- 3.6 Optimization by population annealing Monte Carlo method . . . 30
- 3.7 Analyses by user programs . . . 35

4 Input and output **43**
- 4.1 Input parameters . . . 43
- 4.2 Reference files . . . 46
- 4.3 Output files . . . 47

5 Acknowledgements **49**

6 Contact **51**

CHAPTER

ONE

INTRODUCTION

1.1 What is ODAT-SE ?

ODAT-SE is a framework for applying a search algorithm to a direct problem solver to find the optimal solution. As the standard direct problem solver, the experimental data analysis software for two-dimensional material structure analysis is prepared. The direct problem solver gives the deviation between the experimental data and the calculated data obtained under the given parameters such as atomic positions as a loss function used in the inverse problem. The optimal parameters are estimated by minimizing the loss function using a search algorithm. For further use, the original direct problem solver or the search algorithm can be defined by users. In the current version, for solving a direct problem, ODAT-SE offers the wrapper of the solver for the total-reflection high-energy positron diffraction (TRHEPD), the surface X-ray diffraction (SXRD), and the low-energy electron diffraction (LEED). As algorithms, it offers the Nelder-Mead method, the grid search method, the Bayesian optimization method, the replica exchange Monte Carlo method, and the population annealing Monte Carlo method.

1.2 What is odatse-STR ?

sim-trhepd-rheed has been developed as a simulator software package for the total-reflection high-energy positron diffraction (TRHEPD) and the reflection high energy electron diffraction (RHEED). It takes atom positions as input and calculates the Rocking Curve of the diffraction intensity. The program is written in Fortran and runs on ordinary Linux environment. odatse-STR is an adaptor library to use sim-trhepd-rheed as a direct problem solver of ODAT-SE. It was originally developed as a component of 2DMAT v2.x, and has been restructured as a separate module to be used with ODAT-SE and sim-trhepd-rheed.

1.3 License

This package is distributed under GNU General Public License version 3 (GPL v3).

Copyright (c) <2020-> The University of Tokyo. All rights reserved.

This software was developed with the support of "Project for advancement of software usability in materials science" of The Institute for Solid State Physics, The University of Tokyo. We hope that you cite the following reference when you publish the results using ODAT-SE:

"Data-analysis software framework 2DMAT and its application to experimental measurements for two-dimensional material structures", Y. Motoyama, K. Yoshimi, I. Mochizuki, H. Iwamoto, H. Ichinose, and T. Hoshi, Computer Physics Communications 280, 108465 (2022).

Bibtex:

```
@article{MOTOYAMA2022108465,
  title = {Data-analysis software framework 2DMAT and its application to experimental␣
↪measurements for two-dimensional material structures},
  journal = {Computer Physics Communications},
  volume = {280},
  pages = {108465},
  year = {2022},
  issn = {0010-4655},
  doi = {https://doi.org/10.1016/j.cpc.2022.108465},
  url = {https://www.sciencedirect.com/science/article/pii/S0010465522001849},
  author = {Yuichi Motoyama and Kazuyoshi Yoshimi and Izumi Mochizuki and Harumichi␣
↪Iwamoto and Hayato Ichinose and Takeo Hoshi}
}
```

1.4 Version Information

odatse-STR

- v1.1.0: 2026-10-09
- v1.0.1: 2026-10-09
- v1.0.0: 2024-11-25

ODAT-SE

- v3.0.0: 2024-11-25

2DMAT

- v2.1.0: 2022-04-08
- v2.0.0: 2022-01-17
- v1.0.1: 2021-04-15
- v1.0.0: 2021-03-12
- v0.1.0: 2021-02-08

1.5 Main developers

ODAT-SE, odatse-STR, and 2DMAT has been developed by following members.

- ODAT-SE v3.0.0 -, odat-STR v1.0.0 -
  - Y. Motoyama (The Institute for Solid State Physics, The University of Tokyo)
  - K. Yoshimi (The Institute for Solid State Physics, The University of Tokyo)
  - T. Aoyama (The Institute for Solid State Physics, The University of Tokyo)
  - T. Hoshi (National Institute for Fusion Science)
- 2DMAT v2.0.0 -
  - Y. Motoyama (The Institute for Solid State Physics, The University of Tokyo)
  - K. Yoshimi (The Institute for Solid State Physics, The University of Tokyo)
  - H. Iwamoto (Department of Applied Mathematics and Physics, Tottori University)

  - T. Hoshi (Department of Applied Mathematics and Physics, Tottori University)
- 2DMAT v0.1.0 - v1.0.1
  - Y. Motoyama (The Institute for Solid State Physics, The University of Tokyo)
  - K. Yoshimi (The Institute for Solid State Physics, The University of Tokyo)
  - T. Hoshi (Department of Applied Mathematics and Physics, Tottori University)

CHAPTER

TWO

INSTALLATION OF ODATSE-STR

2.1 Prerequisites

- Python3 (>=3.9)
  - The following Python packages are required.
    - tomli >= 1.2
    - numpy >= 1.14
  - ODAT-SE version 4.0 and later
  - sim-trhepd-rheed version 1.0.2 and later

2.2 How to download and install

1. Install ODAT-SE
   - From source files:

     Download source files of ODAT-SE from the repository as follows:

     ```
     $ git clone https://github.com/issp-center-dev/ODAT-SE.git
     ```

     Install ODAT-SE using `pip` command:

     ```
     $ cd ODAT-SE
     $ python3 -m pip install .
     ```

     You may add `--user` option to install ODAT-SE locally (in `$HOME/.local`).

     If you run the following command instead, optional packages will also be installed at the same time.

     ```
     $ python3 -m pip install .[all]
     ```

2. Install sim-trhepd-rheed
   - Download the source package from the official site:

     ```
     $ wget -O sim-trhepd-rheed-1.0.2.tar.gz https://github.com/sim-trhepd-rheed/sim-
     ↪trhepd-rheed/archive/refs/tags/v1.0.2.tar.gz
     ```

   - Unpack the source package, and compile the source. Edit Makefile in sim-trhepd-rheed/src as needed.

```
$ tar xvfz sim-trhepd-rheed-1.0.2.tar.gz
$ cd sim-trhepd-rheed-1.0.2/src
$ make
```

The executable files `bulk.exe`, `surf.exe`, `potcalc.exe`, and `xyz.exe` will be generated. Put `bulk.exe` and `surf.exe` in a directory listed in the PATH environment variable, or specify the paths to these commands at run time.

3. Install odatse-STR

   - From source files:

     The source files of odatse-STR are available from the GitHub repository. After obtaining the source files, install odatse-STR using `pip` command as follows:

```
$ git clone https://github.com/2DMAT/odatse-STR.git
$ cd odatse-STR
$ python3 -m pip install .
```

     You may add `--user` option to install the package locally (in `$HOME/.local`).

     Then, the library of odatse-STR and the command `odatse-STR` wil be installed.

2.3 How to run

In ODAT-SE, the analysis is done by using a predefined optimization algorithm and a direct problem solver. There are two ways to do analyses of TRHEPD:

1. Use odatse-STR program included in this package to perform analyses. The users prepare an input parameter file in TOML format, and run command with it. The type of the inverse problem algorithms can be chosen by the parameter.
2. Write a program for the analysis with odatse-STR library and ODAT-SE framework. The type of the inverse problem algorithms can be chosen by importing the appropriate module. A flexible use would be possible, for example, to include data generation within the program.

The types of parameters and the instruction to use the library will be given in the subsequent sections.

2.4 How to uninstall

In order to uninstall odatse-STR and ODAT-SE modules, type the following commands:

```
$ python3 -m pip uninstall odatse-STR ODAT-SE
```

CHAPTER

THREE

TUTORIALS

The direct problem solver, `sim_trhepd_rheed`, is based on the Reflection-High-Energy Electron Diffraction (RHEED, TRHEPD) analysis software developed by Prof. Takashi Hanada at Tohoku University.

In TRHEPD, when atomic coordinates are given, diffraction data is given as a simulation result. Therefore, we are dealing with the direct problem from atomic coordinates to diffraction data. On the other hand, in many cases, diffraction data is given experimentally, and the atomic coordinates are required to reproduce the experimental data. These are inverse problems to the above direct problems.

In ODAT-SE, the algorithms for solving the inverse problem can be selected as following algorithms:

- `minsearch`

  Nelder-Mead method.

- `mapper_mpi`

  Searching the entire search grid for a given parameter.

- `bayes`

  Bayesian optimization.

- `exchange`

  Sampling by the replica exchange Monte Carlo method.

- `pamc`

  Sampling by the population annealing Monte Carlo method.

In this tutorial, we will first introduce how to run the direct problem solver `sim-trhepd-rheed`. Then we will instruct how to run `minsearch` , `mapper_mpi`, `bayes`, `exchange`, and `pamc`. Hereinafter, we use `odatse-STR` program included in odatse-STR with input files in TOML format.

At the end of the tutorial, we will explain how to write your own main program for analyses.

3.1 Direct Problem Solver

odatse-STR module provides a wrapper to the forward problem solvers of ODAT-SE for the program sim-trhepd-rheed , which calculates the intensity of reflection fast (positron) electron diffraction (RHEED, TRHEPD) (A. Ichimiya, Jpn. J. Appl. Phys. 22, 176 (1983); 24, 1365 (1985)). In this tutorial, we will present several examples of analyses using algorithms with sim-trhepd-rheed. First, we will explain how to install and test sim-trhepd-rheed. For details, see the official web page for sim-trhepd-rheed.

3.1.1 Download and Install

First, you need to obtain the source files from the repository, and move to the directory of odatse-STR module.

```
$ git clone https://github.com/2DMAT/odatse-STR.git
$ cd odatse-STR
```

Next, you need to download the source files of sim-trhepd-rheed from their repository, and build it.

```
$ git clone https://github.com/sim-trhepd-rheed/sim-trhepd-rheed
$ cd sim-trhepd-rheed/src
$ make
```

`Makefile` should be modified according to your environment. When it is successful, `bulk.exe` and `surf.exe` will be created.

3.1.2 Calculation execution

In sim-trhepd-rheed, the bulk part of the surface structure is first calculated with `bulk.exe`. Then, using the results of the `bulk.exe` calculation (the `bulkP.b` file), the surface portion of the `surf.exe` surface structure is calculated.

In this tutorial, we will actually do the TRHEPD calculation. The sample input files are located in `sample/solver` of odatse-STR. First, copy this folder to a suitable working folder `work`.

```
$ cp -r sample/solver work
$ cd work
```

Next, copy `bulk.exe` and `surf.exe` to `work`.

```
$ cp ../sim-trhepd-rheed/src/bulk.exe .
$ cp ../sim-trhepd-rheed/src/surf.exe .
```

Execute `bulk.exe`.

```
$ ./bulk.exe
```

Then, the bulk file `bulkP.b` will be generated.

```
$ ls
bulk.exe bulk.txt bulkP.b surf.exe surf.txt
```

Next, execute `surf.exe`.

```
$ ./surf.exe
```

After the execution, the files `surf-bulkP.md`, `surf-bulkP.s` and `SURFYYYYMMDD-HHMMSSlog.txt` will be generated. (YYYYMMDD and HHMMSS stand for the execution date and time, respectively.)

3.1.3 Visualization of calculation result

The contents of `surf-bulkP.s` are shown as follow:

```
#azimuths,g-angles,beams
1 56 13
#ih,ik
deg,6 0,5 0,4 0,3 0,2 0,1 0,0 0,-1 0,-2 0,-3 0,-4 0,-5 0,-6 0,
```

(continues on next page)

(continued from previous page)

```
  5.0000E-01,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.
↪0000E+00,  1.5946E-02,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,
↪  0.0000E+00,
  6.0000E-01,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.
↪0000E+00,  1.8701E-02,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,
↪  0.0000E+00,
  7.0000E-01,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.
↪0000E+00,  2.1202E-02,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,
↪  0.0000E+00,
  8.0000E-01,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  2.1711E-
↪03,  1.9273E-02,  2.1711E-03,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.
↪0000E+00,
  9.0000E-01,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  4.3965E-
↪03,  1.7006E-02,  4.3965E-03,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.
↪0000E+00,
  1.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  6.3263E-
↪03,  1.4952E-02,  6.3263E-03,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.
↪0000E+00,
...
```

From the above file, a rocking curve is created whose horizontal axis is the angle (first column of data after row 5) and the vertical axis shows the intensity of the (0,0) peak (eighth column of data after row 5). You can use Gnuplot or other graphing software, but here we use the program `plot_bulkP.py` in the `script` folder. Run it as follows.

```
$ python3 ../script/plot_bulkP.py
```

Then, `plot_bulkP.png` will be created as follows.

We will make convolution and normalization to these diffraction intensity data of the 00 peaks. Prepare `surf-bulkP.s` and run `make_convolution.py`.

```
$ python3 ../script/make_convolution.py
```

When executed, the following file `convolution.txt` will be created.

```
0.500000 0.010818010
0.600000 0.013986716
0.700000 0.016119093
0.800000 0.017039022
0.900000 0.017084666
  ... skipped ...
5.600000 0.000728539
5.700000 0.000530758
5.800000 0.000412908
5.900000 0.000341740
6.000000 0.000277553
```

The first column is the viewing angle, and the second column is the normalized 00-peak diffraction intensity data written in `surf-bulkP.s` with a convolution of half-width 0.5.

By using `plot_convolution.py`, a figure `plot_convolution.png` will be generated as follows.

```
$ python3 ../script/plot_convolution.py
```

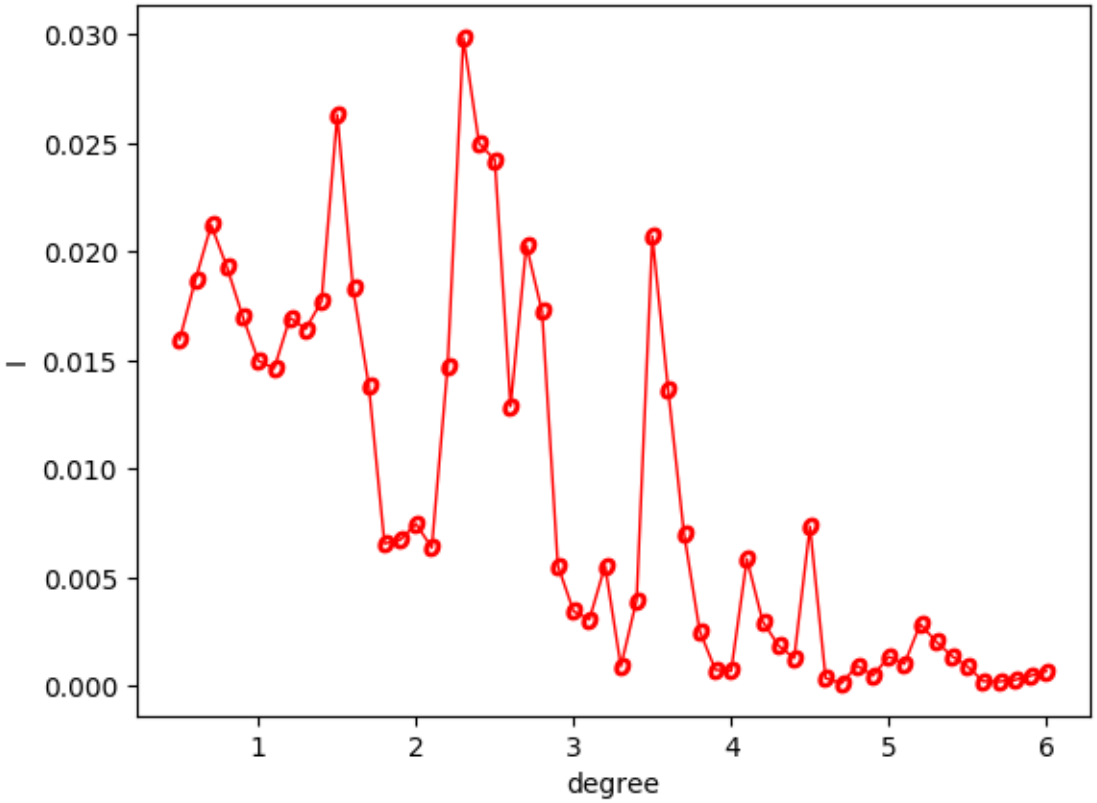


Fig. 3.1: Rocking curve of Si(001)-2x1 surface.

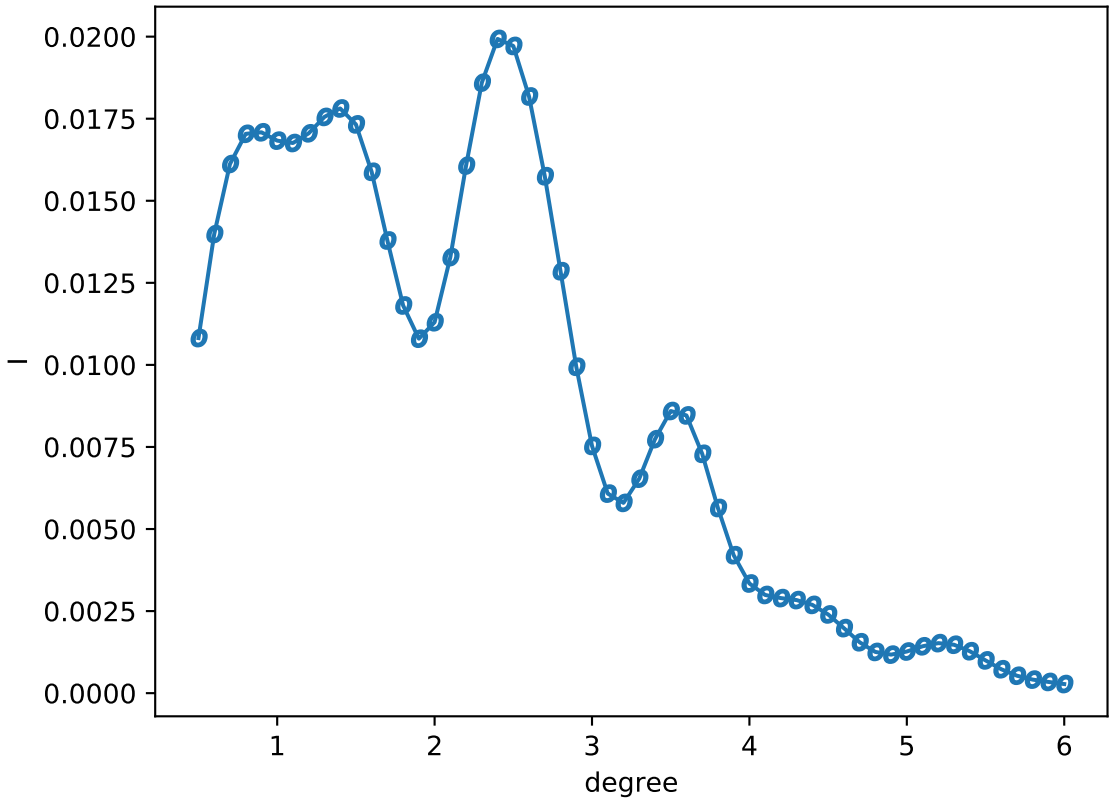


Fig. 3.2: Rocking curve of Si(001)-2x1 surface that is made convolution of half-width 0.5 and normalized.

3.2 Optimization by Nelder-Mead method

In this section, we will explain how to calculate the inverse problem of analyzing atomic coordinates from diffraction data using the Nelder-Mead method. The specific calculation procedure is as follows.

0. Preparation of the reference file

   Prepare the reference file to be matched (in this tutorial, it corresponds to `experiment.txt` described below).

1. Perform calculations on the bulk part of the surface structure.

   Copy `bulk.exe` to `sample/minsearch` and run the calculation.

2. Run the main program

   Run the calculation using `odatse-STR` to estimate the atomic coordinates.

In the main program, the Nelder-Mead method implemented in scipy.optimize.fmin is applied to find the parameter that minimizes the deviation (R-value) between the intensity obtained using the solver (in this case `surf.exe`) and the intensity listed in the reference file (`experiment.txt`).

3.2.1 Location of the sample files

The sample files are located in `sample/single_beam/minsearch`. The following files are stored in the folder.

- `bulk.txt`

  Input file of `bulk.exe`.

- `experiment.txt` , `template.txt`

  Reference file to proceed with calculations in the main program.

- `ref.txt`

  A file containing the answers you want to seek in this tutorial.

- `input.toml`

  Input file of the main program.

- `prepare.sh` , `do.sh`

  Script prepared for doing all calculation of this tutorial

The following sections describe these files and then show the actual calculation results.

3.2.2 The reference file

The `template.txt` file is almost the same format as the input file for `surf.exe`. The parameters to vary (such as the atomic coordinates you want to find) are labeled as `value_*` or some other appropriate strings. The following is the content of `template.txt`.

```
2                                   ,NELMS,  -------- Ge(001)-c4x2
32,1.0,0.1                          ,Ge Z,da1,sap
0.6,0.6,0.6                         ,BH(I),BK(I),BZ(I)
32,1.0,0.1                          ,Ge Z,da1,sap
0.4,0.4,0.4                         ,BH(I),BK(I),BZ(I)
9,4,0,0,2, 2.0,-0.5,0.5              ,NSGS,msa,msb,nsa,nsb,dthick,DXS,DYS
8                                   ,NATM
1, 1.0, 1.34502591  1       value_01   ,IELM(I),ocr(I),X(I),Y(I),Z(I)
1, 1.0, 0.752457792 1       value_02
```

(continues on next page)

(continued from previous page)

```
2, 1.0, 1.480003343 1.465005851     value_03
2, 1.0, 2   1.497500418     2.281675
2, 1.0, 1   1.5     1.991675
2, 1.0, 0   1       0.847225
2, 1.0, 2   1       0.807225
2, 1.0, 1.009998328 1       0.597225
1,1                                  ,(WDOM,I=1,NDOM)
```

In this input file, `value_01`, `value_02`, and `value_03` are used. In the sample folder, there is a reference file `ref.txt` to know if the atomic positions are estimated correctly. The contents of this file are

```
fx = 8.97501627575891e-06
z1 = 5.230573154947887
z2 = 4.3706085364407095
z3 = 3.5961303342247577
```

`value_01`, `value_02`, and `value_03` correspond to `z1`, `z2`, and `z3`, respectively. `fx` is the optimal value of the objective function.

`experiment.txt` is a file that is used as a reference in the main program. In this tutorial, it is equivalent to `convolution.txt` obtained from the calculation when putting the parameters in `ref.txt` into `template.txt` and following the same procedure as in the tutorial on direct problems. (Note that they are different from those in the tutorial of the direct problem because the input files for `bulk.exe` and `surf.exe` are different.)

3.2.3 Input files

In this section, we prepare the input file `input.toml` for the main program. The details can be found in the input file section in the manual. The content of `input.toml` is shown below.

```
[base]
dimension = 3
output_dir = "output"

[solver]
name = "sim-trhepd-rheed"
run_scheme = "subprocess"
generate_rocking_curve = true

[solver.config]
cal_number = [1]

[solver.param]
string_list = ["value_01", "value_02", "value_03" ]

[solver.reference]
path = "experiment.txt"
exp_number = [1]

[solver.post]
normalization = "TOTAL"

[algorithm]
name = "minsearch"
```

(continues on next page)

(continued from previous page)

```
label_list = ["z1", "z2", "z3"]

[algorithm.param]
min_list = [0.0, 0.0, 0.0]
max_list = [10.0, 10.0, 10.0]
initial_list = [5.25, 4.25, 3.50]
```

First, `[base]` section is explained.

- `dimension` is the number of variables to be optimized. In this case it is 3 since we are optimizing three variables as described in `template.txt`.
- `output_dir` is the name of directory for the outputs. If it is omitted, the results are written in the directory in which the program is executed.

`[solver]` section specifies the solver to be used inside the main program and its settings.

- `name` is the name of the solver you want to use. In this tutorial it is `sim-trhepd-rheed`.
- `run_scheme` specifies how the solver is executed within the program. In the current version, `subprocess` can be specified.
- `generate_rocking_curve` specifies whether or not the rocking curves are generated in every steps.

The solver can be configured in the subsections `[solver.config]`, `[solver.param]`, and `[solver.reference]`.

`[solver.config]` section specifies options for reading the output file produced by `surf.exe` that is called from the main program.

- `calculated_first_line` specifies the first line to read from the output file.
- `calculated_last_line` specifies the last line of the output file to be read.
- `cal_number` specifies the indices of columns of the output file to read.

`[solver.param]` section specifies options for the input file passed to `surf.exe` that is to be called from the main program.

- `string_list` is a list of variable names embedded in `template.txt`.

`[solver.reference]` section specifies the location of the experimental data and the range to read.

- `path` specifies the path where the experimental data is located.
- `first` specifies the first line of the experimental data file to read.
- `end` specifies the last line of the experimental data file to read.
- `exp_number` specifies the indices of columns of the experimental data file to read.

`[algorithm]` section specifies the algorithm to use and its settings.

- `name` is the name of the algorithm you want to use. In this tutorial, it is set to `minsearch`, since we are using the Nelder-Mead method.
- `label_list` is a list of label names to be attached to the output of `value_0x` (x=1,2,3).

`[algorithm.param]` section specifies the range of parameters to search and their initial values.

- `min_list` and `max_list` specify the minimum and maximum values of the search range, respectively.
- `initial_list` specifies the initial values.

Other parameters, such as convergence criteria used in the Nelder-Mead method, can be set in the `[algorithm]` section, although they are omitted here so that the default values are used. See the input file section of the manual for details.

3.2.4 Calculation execution

First, move to the folder where the sample files are located. (We assume that you are directly under the directory where you downloaded this software.)

```
$ cd sample/single_beam/minsearch
```

Copy `bulk.exe` and `surf.exe`.

```
$ cp ../../sim-trhepd-rheed/src/bulk.exe .
$ cp ../../sim-trhepd-rheed/src/surf.exe .
```

Run `bulk.exe` to produce `bulkP.b`.

```
$ ./bulk.exe
```

After that, run the main program. The computation time will take only a few seconds on a normal PC.

```
$ odatse-STR input.toml | tee log.txt
```

Then, the standard output will look as follows.

```
name            : minsearch
label_list      : ['z1', 'z2', 'z3']
param.min_list  : [0.0, 0.0, 0.0]
param.max_list  : [10.0, 10.0, 10.0]
param.initial_list: [5.25, 4.25, 3.5]
eval: x=[5.25 4.25 3.5 ], fun=0.015199252435883206
eval: x=[5.22916667 4.3125     3.64583333], fun=0.013702918645281299
eval: x=[5.22569444 4.40625    3.54513889], fun=0.01263527811899235
eval: x=[5.17997685 4.34895833 3.5943287 ], fun=0.006001659635528168
eval: x=[5.17997685 4.34895833 3.5943287 ], fun=0.006001659635528168
eval: x=[5.22066294 4.33260995 3.60408629], fun=0.005145496928704404
eval: x=[5.22066294 4.33260995 3.60408629], fun=0.005145496928704404
eval: x=[5.2185245  4.32627234 3.56743818], fun=0.0032531465329236025
...
```

`x=` are the candidate parameters at each step, and `fun=` is the R-factor value at that point. The results at each step are also written in the folder `output/LogXXXX_YYYY` (where XXXX and YYYY are the step counts). The final estimated parameters will be written to `output/res.dat`. In the current case, the following result will be obtained:

```
z1 = 5.230573154947887
z2 = 4.3706085364407095
z3 = 3.5961303342247577
```

You can see that we will get the same values as the correct answer data in `ref.txt`.

Note that `do.sh` is available as a script for batch calculation. In `do.sh`, `res.txt` and `ref.txt` are also compared for the check. Here is what it does, without further explanation.

```
#!/bin/sh

sh ./prepare.sh

./bulk.exe
```

(continues on next page)

(continued from previous page)

```
time odatse-STR input.toml | tee log.txt

echo diff output/res.txt ref.txt
res=0
diff output/res.txt ref.txt || res=$?
if [ $res -eq 0 ]; then
  echo Test PASS
  true
else
  echo Test FAILED: res.txt and ref.txt differ
  false
fi
```

3.2.5 Visualization of calculation results

If `generate_rocking_curve` is set to true, the data of the rocking curve at each step is stored in `LogXXXX_00000001` (where `XXXX` is the index of steps) as `RockingCurve.txt`. A tool `draw_RC_double.py` is provided to visualize this data. In this section, we will use this tool to visualize the results.

```
$ cp output/0/Log00000000_00000000/RockingCurve.txt RockingCurve_ini.txt
$ cp output/0/Log00000117_00000000/RockingCurve.txt RockingCurve_con.txt
$ cp ../../../script/draw_RC_double.py .
$ python3 draw_RC_double.py
```

By running the above, `RC_double.png` will be generated.

From the figure, we can see that at the last step the result agrees with the experimental data.

3.3 Optimization by Grid search

In this section, we will explain how to perform a grid-type search to analyze atomic coordinates from diffraction data. The grid type search is compatible with MPI. The specific calculation procedure is the same as for `minsearch`. It is necessary to prepare the data file `MeshData.txt` that defines the search grid in advance.

3.3.1 Location of the sample files

The sample files are located in `sample/single_beam/mapper`. The following files are stored in the folder:

- `bulk.txt`

  Input file of `bulk.exe`

- `experiment.txt`, `template.txt`

  Reference file to proceed with calculations in the main program.

- `MeshData.txt`

  Data file for the search grid.

- `input.toml`

  Input file of the main program.

- `ref_ColorMap.txt`

  A file to check if the calculation was performed correctly (the answer to `ColorMap.txt` obtained by doing this tutorial).

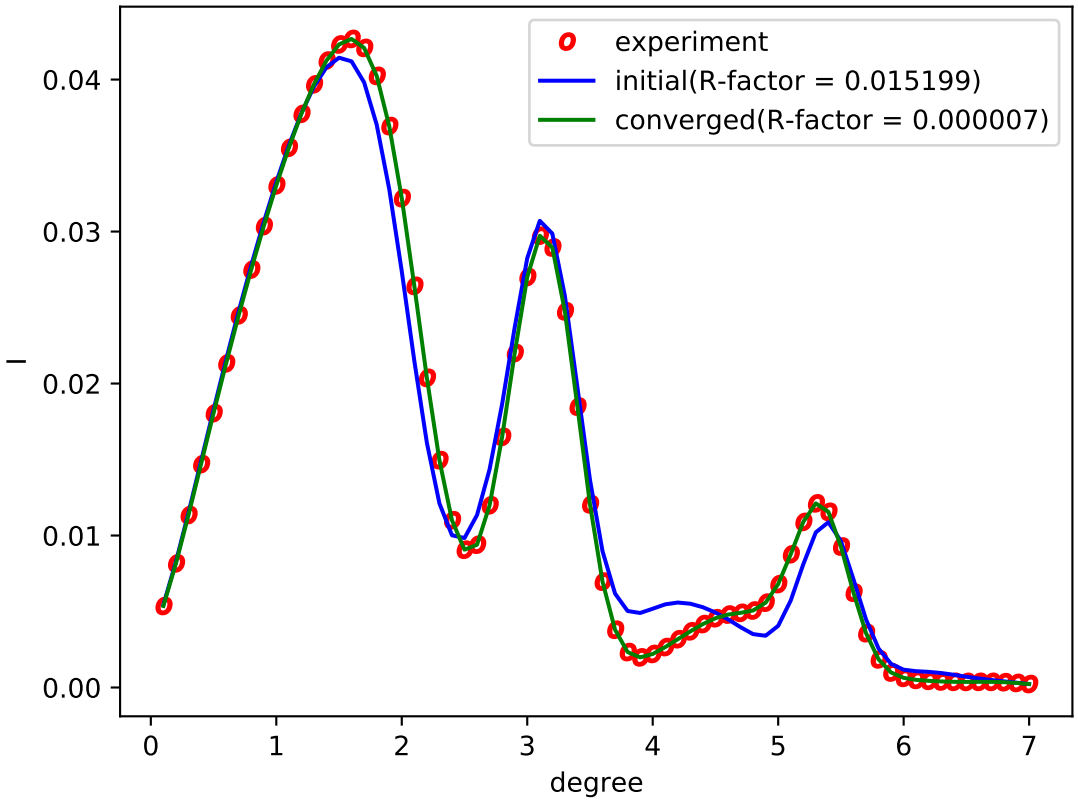


Fig. 3.3: Analysis using the Nelder-Mead method. The red circles represent the experimental values, the blue line represents the rocking curve at the first step, and the green line represents the rocking curve obtained at the last step.

- `prepare.sh`, `do.sh`

  Script prepared for bulk calculation of this tutorial.

Below, we will describe these files and then show the actual calculation results.

3.3.2 Reference files

The `template.txt` and `experiment.txt` are the same as in the previous tutorial (Nealder-Mead optimization). Except, to reduce the computation time, the value of `value_03` is fixed to `3.5` instead of the variable, and thus the grid is searched in two-dimensional space. The actual grid to be searched is given in `MeshData.txt`. In the sample, the contents of `MeshData.txt` are as follows.

```
1 6.000000 6.000000
2 6.000000 5.750000
3 6.000000 5.500000
4 6.000000 5.250000
5 6.000000 5.000000
6 6.000000 4.750000
7 6.000000 4.500000
8 6.000000 4.250000
9 6.000000 4.000000
...
```

The first column is the serial number, and the second and subsequent columns are the values of `value_0`, `value_1`, and so on that are referred to by the variables in `template.txt` in that order.

3.3.3 Input file

This section describes the input file for the main program, `input.toml`. The details of `input.toml` can be found in the input file section of the manual. The following is the content of `input.toml` in the sample file.

```
[base]
dimension = 2
output_dir = "output"

[solver]
name = "sim-trhepd-rheed"
run_scheme = "subprocess"
generate_rocking_curve = false

[solver.config]
cal_number = [1]

[solver.param]
string_list = ["value_01", "value_02" ]

[solver.post]
normalization = "TOTAL"

[solver.reference]
path = "experiment.txt"
exp_number = [1]

[algorithm]
```

(continues on next page)

(continued from previous page)

```
name = "mapper"
label_list = ["z1", "z2"]

[algorithm.param]
mesh_path = "./MeshData.txt"
```

First, [base] section is explained.

- dimension is the number of variables to be optimized. In this case, it is 2 since we are optimizing two variables as described in template.txt.
- output_dir is the name of directory for the outputs. If it is omitted, the results are written in the directory in which the program is executed.

[solver] section specifies the solver to be used inside the main program and its settings.

- name is the name of the solver you want to use. In this tutorial it is sim-trhepd-rheed.
- run_scheme specifies how the solver is executed within the program. In the current version, subprocess can be specified.

The solver can be configured in the subsections [solver.config], [solver.param], and [solver.reference].

[solver.config] section specifies options for reading the output file produced by surf.exe that is called from the main program.

- calculated_first_line specifies the first line to read from the output file.
- calculated_last_line specifies the last line of the output file to be read.
- cal_number specifies the indices of columns of the output file to read.

[solver.param] section specifies options for the input file passed to surf.exe that is to be called from the main program.

- string_list is a list of variable names embedded in template.txt.

[solver.reference] section specifies the location of the experimental data and the range to read.

- path specifies the path where the experimental data is located.
- first specifies the first line of the experimental data file to read.
- end specifies the last line of the experimental data file to read.
- exp_number specifies the indices of columns of the experimental data file to read.

[algorithm] section specifies the algorithm to use and its settings.

- name is the name of the algorithm you want to use. In this tutorial we will use mapper since we will be using grid-search method.
- label_list is a list of label names to be attached to the output of value_0x (x=1,2).

For details on other parameters that can be specified in the input file, please see the Input File section of the manual.

3.3.4 Calculation execution

First, move to the folder where the sample files are located. (We assume that you are directly under the directory where you downloaded this software.)

```
$ cd sample/single_beam/mapper
```

Copy bulk.exe and surf.exe.

```
$ cp ../../sim-trhepd-rheed/src/bulk.exe .
$ cp ../../sim-trhepd-rheed/src/surf.exe .
```

Run `bulk.exe` to produce `bulkP.b`.

```
$ ./bulk.exe
```

After that, run the main program. The computation time will take only a few seconds on a normal PC.

```
$ mpiexec -np 2 odatse-STR input.toml | tee log.txt
```

Here, the calculation using MPI parallel with 2 processes will be done.

When executed, a folder for each rank will be created, and a subfolder `LogXXXX_YYYY` (where `XXXX` and `YYYY` are the grid id and the sequence number, respectively) will be created under it. (The grid id is associated to the index in `MeshData.txt`.) The standard output will look like as follows.

```
name            : mapper
label_list      : ['z1', 'z2']
param.mesh_path : ./MeshData.txt
Iteration : 1/66
Iteration : 2/66
Iteration : 3/66
Iteration : 4/66
Iteration : 5/66
Iteration : 6/66
...
Iteration : 63/66
Iteration : 64/66
Iteration : 65/66
Iteration : 66/66
[0] minimum_value: 1.51992524e-02 at [5.25, 4.25] (mesh 34)
Make ColorMap
complete main process : rank 00000000/00000001
end of run
```

Finally, the `R-factor` calculated at all the points on the grid will be written to `ColorMap.txt`. In this case, the following results will be obtained.

```
6.000000 6.000000 0.047852
6.000000 5.750000 0.055011
6.000000 5.500000 0.053190
6.000000 5.250000 0.038905
6.000000 5.000000 0.047674
6.000000 4.750000 0.065919
6.000000 4.500000 0.053675
6.000000 4.250000 0.061261
6.000000 4.000000 0.069351
6.000000 3.750000 0.071868
6.000000 3.500000 0.072739
...
```

The first and second columns contain the values of `value_01` and `value_02`, and the third column contains the `R-factor`.

Note that `do.sh` is available as a script for batch calculation. In `do.sh`, `res.txt` and `ref.txt` are also compared for the check. Here is what it does, without further explanation.

```
#!/bin/sh

sh prepare.sh

./bulk.exe

time mpiexec -np 2 odatse-STR input.toml

echo diff output/ColorMap.txt ref_ColorMap.txt
res=0
diff output/ColorMap.txt ref_ColorMap.txt || res=$?
if [ $res -eq 0 ]; then
  echo TEST PASS
  true
else
  echo TEST FAILED: ColorMap.txt and ref_ColorMap.txt differ
  false
fi
```

3.3.5 Visualization of calculation results

By examining `ColorMap.txt`, we can estimate the region where the value of `R-factor` becomes small. In this case, the following command will create a two-dimensional parameter space diagram `ColorMapFig.png`.

```
$ python3 plot_colormap_2d.py
```

Looking at the generated figure, we can see that it has the minimum value around (5.25, 4.25).

When `generate_rocking_curve` in `[solver]` section is set to true, `RockingCurve_calculated.txt` will be generated in each log directory. By using it, you can compare the results with the experimental values following the procedure in the previous tutorial.

3.4 Optimization by Bayesian Optimization

This tutorial describes how to estimate atomic positions from the experimental diffraction data by using Bayesian optimization (BO). odatse-STR uses PHYSBO for BO. The search grid has to be prepared in advance as a data file `MeshData.txt`, similar to the case of the grid search.

3.4.1 Sample files

Sample files are available from `sample/single_beam/bayes` . This directory includes the following files:

- `bulk.txt`

  The input file of `bulk.exe`.

- `experiment.txt`, `template.txt`

  Reference files for the main program.

- `MeshData.txt`

  The data file for the search grid.

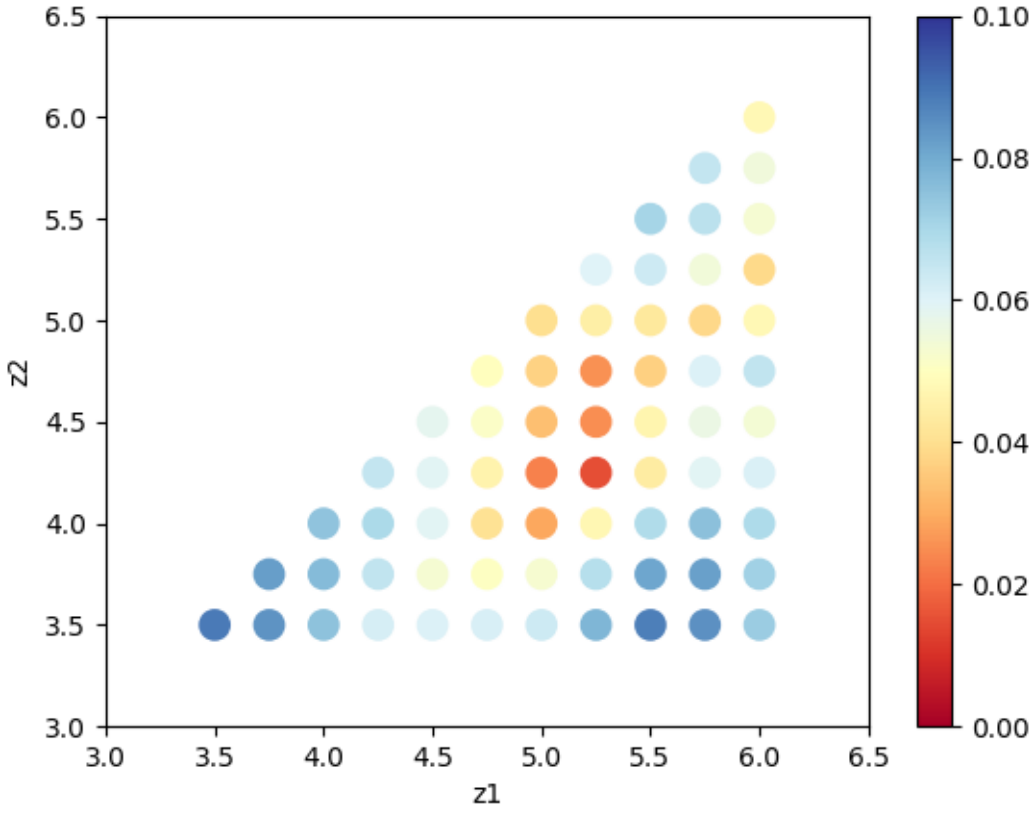


Fig. 3.4: `R-factor` on a two-dimensional parameter space.

- `ref_BayesData.txt`

  Solution file for checking whether the calucation successes or not.

- `input.toml`

  The input file of odatse-STR.

- `prepare.sh`, `do.sh`

  Script files for running this tutorial.

In the following, we will subscribe these files and then show the result.

3.4.2 Reference files

This tutorial uses `template.txt` and `experiment.txt` that are similar to those in the previous tutorial (`minsearch`). Except, in this tutorial the third parameter `value_03` is fixed to `3.5` in order to speed up the calculation. The parameter space to be explored is given by `MeshData.txt`.

```
1 3.5 3.5
2 3.6 3.5
3 3.6 3.6
4 3.7 3.5
5 3.7 3.6
6 3.7 3.7
7 3.8 3.5
8 3.8 3.6
9 3.8 3.7
10 3.8 3.8
...
```

The first column is the index of the point, and the remaining columns are the coodinates, `value_0` and `value_1` that appear in the `template.txt`.

3.4.3 Input files

This subsection describes the input file. For details, see Bayes optimization section of ODAT-SE manual. `input.toml` in the sample directory is shown as the following:

```
[base]
dimension = 2
output_dir = "output"

[solver]
name = "sim-trhepd-rheed"
run_scheme = "subprocess"
generate_rocking_curve = false

[solver.config]
cal_number = [1]

[solver.param]
string_list = ["value_01", "value_02" ]

[solver.reference]
path = "experiment.txt"
```

(continues on next page)

(continued from previous page)

```
exp_number = [1]

[algorithm]
name = "bayes"
label_list = ["z1", "z2"]
seed = 1

[algorithm.param]
mesh_path = "MeshData.txt"

[algorithm.bayes]
random_max_num_probes = 10
bayes_max_num_probes = 20
```

`[base]` section describes the settings for a whole calculation.

- `dimension` is the number of variables you want to optimize. In this case, `2` is specified because two variables are optimized.
- `output_dir` is the name of directory for the outputs. If it is omitted, the results are written in the directory in which the program is executed.

`[solver]` section specifies the solver to use inside the main program and its settings.

See the minsearch tutorial.

`[algorithm]` section sets the algorithm to use and its settings.

- `name` is the name of the algorithm you want to use. In this tutorial we will do a Bayesian optimization analysis, and so it is set to `bayes`.
- `label_list` is a list of labels shown in the output of the values of `value_0x` (x = 1,2).

`[algorithm.bayes]` section sets the parameters for Bayesian optimization.

- `random_max_num_probes` specifies the number of random searches before Bayesian optimization.
- `bayes_max_num_probes` specifies the number of Bayesian searches.

For details on other parameters that can be specified in the input file, see the chapter on input files of `bayes`.

3.4.4 Calculation

First, move to the folder where the sample file is located. (Hereinafter, it is assumed that you are the root directory of odatse-STR.)

```
$ cd sample/single_beam/bayes
```

Copy `bulk.exe` and `surf.exe` as in the tutorial for the direct problem.

```
$ cp ../../sim-trhepd-rheed/src/bulk.exe .
$ cp ../../sim-trhepd-rheed/src/surf.exe .
```

Run `bulk.exe` to generate `bulkP.b` .

```
$ ./bulk.exe
```

Then, run the main program. It will take a few secondes on a normal PC.

```
$ python3 odatse-STR input.toml | tee log.txt
```

A directory `output/0` will be created. The following standard output will be shown:

```
# parameter
random_max_num_probes = 10
bayes_max_num_probes = 20
score = TS
interval = 5
num_rand_basis = 5000
name            : bayes
label_list      : ['z1', 'z2']
seed            : 1
param.mesh_path : ./MeshData.txt
bayes.random_max_num_probes: 10
bayes.bayes_max_num_probes: 20
0001-th step: f(x) = -0.037237 (action=150)
   current best f(x) = -0.037237 (best action=150)

0002-th step: f(x) = -0.060508 (action=36)
   current best f(x) = -0.037237 (best action=150)

0003-th step: f(x) = -0.062158 (action=175)
   current best f(x) = -0.037237 (best action=150)

0004-th step: f(x) = -0.049211 (action=85)
   current best f(x) = -0.037237 (best action=150)

0005-th step: f(x) = -0.083945 (action=255)
   current best f(x) = -0.037237 (best action=150)

0006-th step: f(x) = -0.055569 (action=170)
   current best f(x) = -0.037237 (best action=150)
...
```

where a list of hyperparameters are shown, followed by the candidate parameters at each step and the corresponding `R-factor` multiplied by -1. It also outputs the grid index (`action`) and `f(x)` with the best `R-factor` at that time.

Under the directory `output/0`, subdirectories `LogXXXX_00000000` are created where XXXX is the grid index and the solver outputs are stored for each grid. (The first column in `MeshData.txt` is taken as the id of the grid.) The final estimated parameters are output to `BayesData.txt`.

In this case, `BayesData.txt` can be seen as the following

```
#step z1 z2 fx z1_action z2_action fx_action
0 5.1 4.9 0.037237314010261195 5.1 4.9 0.037237314010261195
1 5.1 4.9 0.037237314010261195 4.3 3.5 0.06050786306685965
2 5.1 4.9 0.037237314010261195 5.3 3.9 0.06215778000834068
3 5.1 4.9 0.037237314010261195 4.7 4.2 0.049210767760634364
4 5.1 4.9 0.037237314010261195 5.7 3.7 0.08394457854191653
5 5.1 4.9 0.037237314010261195 5.2 5.2 0.05556857782716691
6 5.1 4.9 0.037237314010261195 5.7 4.0 0.0754639895013157
7 5.1 4.9 0.037237314010261195 6.0 4.4 0.054757310814479355
8 5.1 4.9 0.037237314010261195 6.0 4.2 0.06339787375966344
```

(continues on next page)

(continued from previous page)

```
9 5.1 4.9 0.037237314010261195 5.7 5.2 0.05348404677676544
10 5.1 4.7 0.03002813055356341 5.1 4.7 0.03002813055356341
11 5.1 4.7 0.03002813055356341 5.0 4.4 0.03019977423448576
12 5.3 4.5 0.02887504880071686 5.3 4.5 0.02887504880071686
13 5.1 4.5 0.025865346123665988 5.1 4.5 0.025865346123665988
14 5.2 4.4 0.02031077875240244 5.2 4.4 0.02031077875240244
15 5.2 4.4 0.02031077875240244 5.2 4.6 0.023291891689059388
16 5.2 4.4 0.02031077875240244 5.2 4.5 0.02345999725278686
17 5.2 4.4 0.02031077875240244 5.1 4.4 0.022561543431398066
18 5.2 4.4 0.02031077875240244 5.3 4.4 0.02544527153306051
19 5.2 4.4 0.02031077875240244 5.1 4.6 0.02778877135528466
20 5.2 4.3 0.012576357659158034 5.2 4.3 0.012576357659158034
21 5.1 4.2 0.010217361468113488 5.1 4.2 0.010217361468113488
22 5.1 4.2 0.010217361468113488 5.2 4.2 0.013178053637167673
...
```

The first column contains the number of steps, and the second, third, and fourth columns contain `value_01`, `value_02`, and `R-factor`, which give the highest score at that time. These are followed by the candidate `value_01`, `value_02` and `R-factor` for that step. In this case, you can see that the correct solution is obtained at the 21th step.

Note that `do.sh` is prepared as a script for batch calculation. `do.sh` also checks the difference between `BayesData.dat` and `ref_BayesData.dat`. The script is read as follows, though we omit further explanation.

```
#!/bin/sh

sh prepare.sh

./bulk.exe

time odatse-STR input.toml

echo diff output/BayesData.txt ref_BayesData.txt
res=0
diff output/BayesData.txt ref_BayesData.txt || res=$?
if [ $res -eq 0 ]; then
  echo TEST PASS
  true
else
  echo TEST FAILED: BayesData.txt.txt and ref_BayesData.txt.txt differ
  false
fi
```

3.4.5 Visualization

You can see at which step the parameter gave the minimum score from `BayesData.txt`. Since `RockingCurve.txt` is stored in the subfolder for each step, it is possible to compare it with the experimental value by following the procedure in :doc:`minsearch`.

3.5 Optimization by replica exchange Monte Carlo method

This tutorial describes how to estimate atomic positions from the experimental diffraction data by using the replica exchange Monte Carlo method (RXMC).

3.5.1 Sample files

Sample files are available from `sample/single_beam/exchange` . This directory includes the following files:

- `bulk.txt`

  The input file of `bulk.exe`.

- `experiment.txt`, `template.txt`

  Reference files for the main program.

- `ref.txt`

  Solution file for checking whether the calucation is successful or not.

- `input.toml`

  The input file of the main program.

- `prepare.sh`, `do.sh`

  Script files for running this tutorial.

In the following, we will describe these files and then show the result.

3.5.2 Reference files

This tutorial uses the reference files, `template.txt` and `experiment.txt`, which are the same as those used in the previous tutorial (*Optimization by Nelder-Mead method*).

3.5.3 Input files

This subsection describes the input files. For details, see the replica exchange Monte Carlo method in ODAT-SE manual. `input.toml` in the sample directory is shown as the following:

```
[base]
dimension = 2
output_dir = "output"

[algorithm]
name = "exchange"
label_list = ["z1", "z2"]
seed = 12345

[algorithm.param]
min_list = [3.0, 3.0]
max_list = [6.0, 6.0]
step_list = [1.0, 1.0]

[algorithm.exchange]
numsteps = 1000
numsteps_exchange = 20
Tmin = 0.005
```

(continues on next page)

(continued from previous page)

```
Tmax = 0.05
Tlogspace = true

[solver]
name = "sim-trhepd-rheed"
run_scheme = "subprocess"

[solver.config]
cal_number = [1]

[solver.param]
string_list = ["value_01", "value_02" ]
degree_max = 7.0

[solver.reference]
path = "experiment.txt"
exp_list = [1]

[solver.post]
normalization = "TOTAL"
```

In the following, we will briefly describe the contents of the file. For details, see the algorithm section of ODAT-SE manual.

`[base]` section describes the settings for a whole calculation.

- `dimension` is the number of variables you want to optimize. In this case, specify `2` because it optimizes two variables.
- `output_dir` is the name of directory for the outputs. If it is omitted, the results are written in the directory in which the program is executed.

`[solver]` section specifies the solver to use inside the main program and its settings.

- See the minsearch tutorial.

`[algorithm]` section sets the algorithm to use and its settings.

- `name` is the name of the algorithm you want to use. In this tutorial we will use RXMC, so specify `exchange`.
- `label_list` is a list of labels to be attached to the output of `value_0x` (x = 1,2).
- `seed` is the seed that a pseudo-random number generator uses.

`[algorithm.param]` section sets the parameter space to be explored.

- `min_list` is a lower bound and `max_list` is an upper bound. `step_list` specifies the step size of one Monte Carlo update (deviation of Gaussian).

`[algorithm.exchange]` section sets the parameters for RXMC.

- `numstep` is the number of Monte Carlo steps.
- `numsteps_exchange` is the number of interval steps between temperature exchanges.
- `Tmin`, `Tmax` are the minimum and the maximum of temperature, respectively.
- When `Tlogspace` is `true`, the temperature points are distributed uniformly in the logarithmic space.

`[solver]` section specifies the solver to use inside the main program and its settings.

- See the *Optimization by Nelder-Mead method* tutorial.

3.5.4 Calculation

First, move to the folder where the sample file is located. (Hereinafter, it is assumed that you are the root directory of odatse-STR.)

```
$ cd sample/single_beam/exchange
```

Copy `bulk.exe` and `surf.exe` as in the tutorial for the direct problem.

```
$ cp ../../sim-trhepd-rheed/src/bulk.exe .
$ cp ../../sim-trhepd-rheed/src/surf.exe .
```

Execute `bulk.exe` to generate `bulkP.b` .

```
$ ./bulk.exe
```

Then, run the main program. It will take a few secondes on a normal PC.

```
$ mpiexec -np 4 odatse-STR input.toml | tee log.txt
```

Here, the calculation is performed using MPI parallel with 4 processes. If you are using Open MPI and you request more processes than the number of available CPU cores, add the `--oversubscribed` option to the `mpiexec` command.

When executed, a folder for each rank will be created under the directory `output`, and `trial.txt` and `result.txt` will be created. `trial.txt` contains the parameters evaluated in each Monte Carlo step and the value of the objective function, and `result.txt` contains the parameters actually adopted.

These files have the same format: the first two columns are time (step) and the index of walker in the process, the third is the temperature, the fourth column is the value of the objective function, and the fifth and subsequent columns are the parameters.

```
# step walker T fx x1 x2
0 0 0.004999999999999999 0.07830821484593968 3.682008067401509 3.9502750191292586
1 0 0.004999999999999999 0.07830821484593968 3.682008067401509 3.9502750191292586
2 0 0.004999999999999999 0.07830821484593968 3.682008067401509 3.9502750191292586
3 0 0.004999999999999999 0.06273922648753057 4.330900869594549 4.311333132184154
```

In the case of the sim-trhepd-rheed solver, a subfolder `LogXXXX_YYYY` (`XXXX` is the number of MC steps) is created under each working directory, and the rocking curve information and other outputs are recorded.

`result.txt` in the output directory for each MPI rank records the data sampled by each replica. They are rearranged according to the temperature, and stored in the files `output/result_T*.txt` in which * stands for the index of the temperature.

Finally, `best_result.txt` is filled with the information about the parameters with the value of the optimal objective function (R-factor), the rank from which it was obtained, and the Monte Carlo step.

```
nprocs = 4
rank = 1
step = 282
walker = 0
fx = 0.008414800224430936
z1 = 5.164773671165013
z2 = 4.226467514644945
```

In addition, `do.sh` is prepared as a script for batch calculation. `do.sh` also checks the difference between `best_result.txt` and `ref.txt`. The content of the script is shown below, though further information will be omitted.

```
#!/bin/sh

sh prepare.sh

./bulk.exe

time mpiexec --oversubscribe -np 4 odatse-STR input.toml

echo diff output/best_result.txt ref.txt
res=0
diff best_result.txt ref.txt || res=$?
if [ $res -eq 0 ]; then
  echo TEST PASS
  true
else
  echo TEST FAILED: best_result.txt and ref.txt differ
  false
fi
```

3.5.5 Visualization

By illustrating `output/result_T*.txt`, you can estimate regions where the parameters with small R-factor are. In this case, the figure `result.png` of the 2D parameter space is created for the data in `output/result_T1.txt` by using the following command.

```
$ python3 plot_result_2d.py
```

Looking at the resulting diagram, we can see that the samples are concentrated near (5.25, 4.25) and (4.25, 5.25), and that the `R-factor` value is small there.

Also, `RockingCurve.txt` is stored in each subfolder `LogXXXX_YYYY` (`XXXX` is the index of MC step and `YYYY` is the index of replica in the MPI process) when `generate_rocking_curve` in `[solver]` section is set to true. By using this, it is possible to compare the result with the experimental value according to the procedure of the previous tutorial.

3.6 Optimization by population annealing Monte Carlo method

This tutorial describes how to estimate atomic positions from the experimental diffraction data by using the population annealing Monte Carlo method (PAMC).

3.6.1 Sample files

Sample files are available from `sample/single_beam/pamc` . This directory includes the following files:

- `bulk.txt`

  The input file of `bulk.exe`.

- `experiment.txt`, `template.txt`

  Reference files for the main program.

- `ref.txt`

  Solution file for checking whether the calucation is successful or not (reference for `fx.txt`).

- `input.toml`

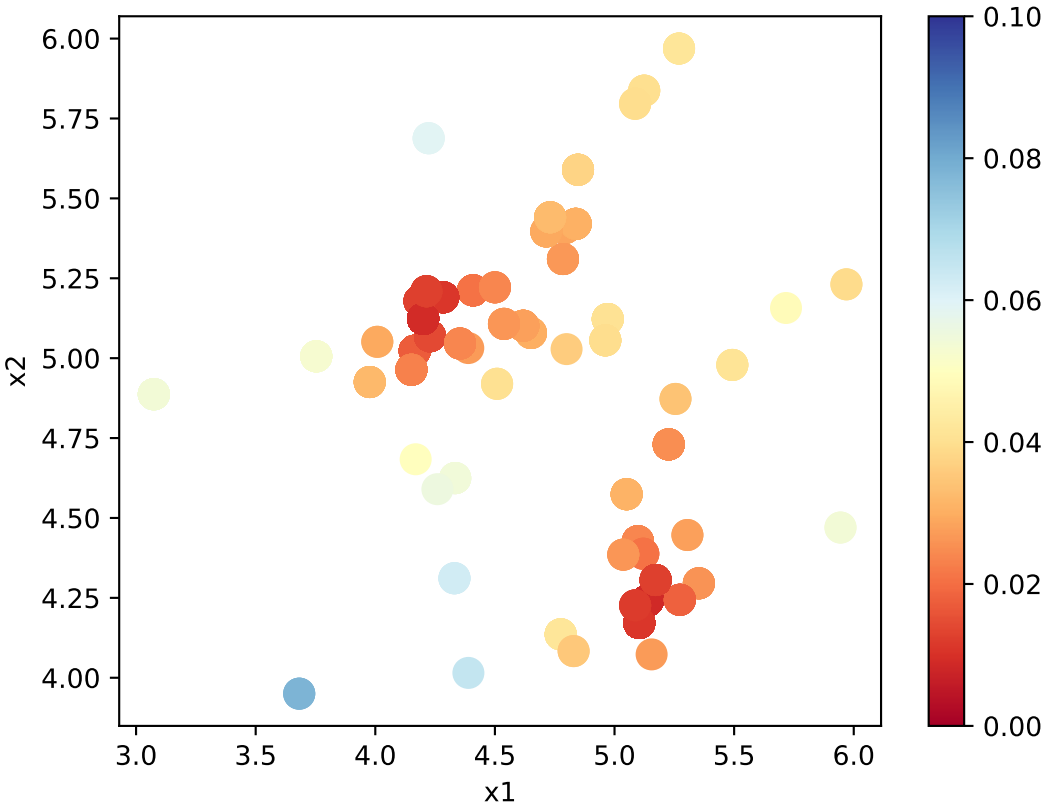


Fig. 3.5: Sampled parameters and `R-factor`. The horizontal axes is x1 (`value_01`) and the vertical axes is x2 (`value_02`).

The input file of odatse-STR.

- `prepare.sh`, `do.sh`

  Script files for running this tutorial.

In the following, we will subscribe these files and then show the result.

3.6.2 Reference files

This tutorial uses the reference files, `template.txt` and `experiment.txt`, which are the same as those used in the previous tutorial (*Optimization by Nelder-Mead method*).

3.6.3 Input files

This subsection describes the input file. For details, see the population annealing Monte Carlo method in ODAT-SE manual. `input.toml` in the sample directory is shown as the following:

```
[base]
dimension = 2
output_dir = "output"

[algorithm]
name = "pamc"
label_list = ["z1", "z2"]
seed = 12345

[algorithm.param]
min_list = [3.0, 3.0]
max_list = [6.0, 6.0]
step_list = [0.3, 0.3]

[algorithm.pamc]
numsteps_annealing = 5
bmin = 0.0
bmax = 200.0
Tnum = 21
Tlogspace = false
nreplica_per_proc = 10

[solver]
name = "sim-trhepd-rheed"
run_scheme = "subprocess"

[solver.config]
cal_number = [1]

[solver.param]
string_list = ["value_01", "value_02" ]

[solver.post]
normalization = "TOTAL"

[solver.reference]
```

(continues on next page)

(continued from previous page)

```
path = "experiment.txt"
exp_number = [1]
```

In the following, we will briefly describe this input file. For details, see the manual of the population annealing Monte Carlo method in ODAT-SE manual.

`[base]` section describes the settings for a whole calculation.

- `dimension` is the number of variables you want to optimize. In this case, specify `2` because it optimizes two variables.
- `output_dir` is the name of directory for the outputs. If it is omitted, the results are written in the directory in which the program is executed.

`[algorithm]` section sets the algorithm to use and its settings.

- `name` is the name of the algorithm you want to use. In this tutorial we will use PAMC, so specify `pamc`.
- `label_list` is a list of labels to be attached to the output of `value_0x` (x = 1,2).
- `seed` is the seed that a pseudo-random number generator uses.

`[algorithm.param]` section sets the parameter space to be explored.

- `min_list` is a lower bound and `max_list` is an upper bound.
- `unit_list` is step length in one MC update (deviation of Gaussian)

`[algorithm.pamc]` section sets the parameters for PAMC.

- `numsteps_annealing` is the number of interval steps between temperature decreasing.
- `bmin`, `bmax` are the minimum and the maximum of inversed temperature, respectively.
- `Tnum` is the number of (inversed) temperature points.
- When `Tlogspace` is `true`, the temperature points are distributed uniformly in the logarithmic space.
- `nreplica_per_proc` is the number of replicas (MC walkers) in one MPI process.

`[solver]` section specifies the solver to use inside the main program and its settings.

- See the *Optimization by Nelder-Mead method* tutorial.

3.6.4 Calculation

First, move to the folder where the sample file is located. (Hereinafter, it is assumed that you are the root directory of odatse-STR.)

```
$ cd sample/single_beam/pamc
```

Copy `bulk.exe` and `surf.exe` as in the tutorial for the direct problem.

```
$ cp ../../sim-trhepd-rheed/src/bulk.exe .
$ cp ../../sim-trhepd-rheed/src/surf.exe .
```

Execute `bulk.exe` to generate `bulkP.b`.

```
$ ./bulk.exe
```

Then, run the main program. It will take a few secondes on a normal PC.

```
$ mpiexec -np 4 odatse-STR input.toml | tee log.txt
```

Here, the calculation is performed using MPI parallel with 4 processes. If you are using OpenMPI and you request more processes than the number of available CPU cores, add the `--oversubscribed` option to the `mpiexec` command.

When executed, a folder for each MPI rank will be created under the directory `output`. There, `trial_TXXX.txt` and `result_TXXX.txt` will be created (where `XXX` is the index of points). The former contains the parameters evaluated in each Monte Carlo step and the value of the objective function at each temperature, and the latter contains the parameters actually adopted. These files are concatnated into `trial.txt` and `result.txt`, respectively.

These files have the same format: the first two columns are time (step) and the index of walker in the process, the third is the (inversed) temperature, the fourth column is the value of the objective function, and the fifth and subsequent columns are the parameters. The final two columns are the weight of walker (Neal-Jarzynski weight) and the index of the grand ancestor (the replica index at the beginning of the calculation).

```
# step walker beta fx z1 z2 weight ancestor
0 0 0.0 0.07702743614780189 5.788848278451443 3.949126663745358 1.0 0
0 1 0.0 0.08737730661436376 3.551756435031283 3.6136808356591192 1.0 1
0 2 0.0 0.04954470587051104 4.70317508724506 4.786634108937754 1.0 2
0 3 0.0 0.04671675601156148 5.893543559206865 4.959531290614713 1.0 3
0 4 0.0 0.04142014655238446 5.246719912601735 4.960709612555206 1.0 4
```

In the case of the sim-trhepd-rheed solver, a subfolder `LogXXXX_YYYY` (`XXXX` is the index of MC steps) is created under each working directory, and the rocking curve information and other outputs are recorded.

`best_result.txt` is filled with the information about the parameters with the value of the optimal objective function (R-factor), the rank from which it was obtained, and the Monte Carlo step.

```
nprocs = 4
rank = 0
step = 71
walker = 5
fx = 0.008186713312593607
z1 = 4.225633749839847
z2 = 5.142666117413409
```

Finally, `fx.txt` stores the statistics at each temperature point:

```
# $1: 1/T
# $2: mean of f(x)
# $3: standard error of f(x)
# $4: number of replicas
# $5: log(Z/Z0)
# $6: acceptance ratio
0.0 0.06428002079611472 0.002703413400677839 40 0.0 0.795
10.0 0.061399304916174735 0.002649424392996749 40 -0.6280819199879947 0.85
20.0 0.05904248889111052 0.003162271121295203 4 40 -1.2283060742855603 0.74
30.0 0.04956921148431115 0.0028298565759159633 40 -1.7991035905899855 0.67
...
```

The first column is (inversed) temperature, and the second/third ones are the mean and standard error of $f(x)$, respectively. The fourth column is the number of replicas and the fifth one is the logarithm of the ratio of the partition functions, $\log(Z_n/Z_0)$, where Z_0 is the partition function at the first temperature. The sixth column is the acceptance ratio of MC updates.

In addition, `do.sh` is prepared as a script for batch calculation. `do.sh` also checks the difference between `best_result.txt` and `ref.txt`. The content of the script is shown below, though further information will be omitted.

```
#!/bin/sh

sh prepare.sh

./bulk.exe

time mpiexec --oversubscribe -np 4 odatse-STR input.toml

echo diff output/fx.txt ref.txt
res=0
diff output/fx.txt ref.txt || res=$?
if [ $res -eq 0 ]; then
  echo TEST PASS
  true
else
  echo TEST FAILED: output/fx.txt and ref.txt differ
  false
fi
```

3.6.5 Visualization

By illustrating `result_T.txt`, you can estimate regions where the parameters with small R-factor are. In this case, the figure `result_fx.png` and `result_T.png` of the 2D parameter space is created by using the following command. The color of symbols of `result_fx.png` and `result_T.png` mean `R-factor` and β, respectively.

```
$ python3 plot_result_2d.py
```

Looking at the resulting diagram, we can see that the samples are concentrated near (5.25, 4.25) and (4.25, 5.25), and that the `R-factor` value is small there.

When `generate_rocking_curve` in `[solver]` section is set to true, `RockingCurve.txt` is stored in each subfolder, `LogXXXX_YYYY` (`XXXX` is an index of MC step and `YYYY` is an index of a replica in the MPI process). By using this, it is possible to compare with the experimental value according to the procedure of the previous tutorial.

3.7 Analyses by user programs

In this tutorial, we will write a user program using odatse-STR module and perform analyese. As an example, we adopt Nelder-Mead method for the inverse problem algorithm.

3.7.1 Location of the sample files

The sample files are located in `sample/single_beam/user_program`. The following files are stored in the folder.

- `simple.py`

  Source file of the main program. This program reads `input.toml` for the parameters.

- `input.toml`

  Input file of the main program.

- `experiment.txt`, `template.txt`

  Reference file to proceed with calculations in the main program.

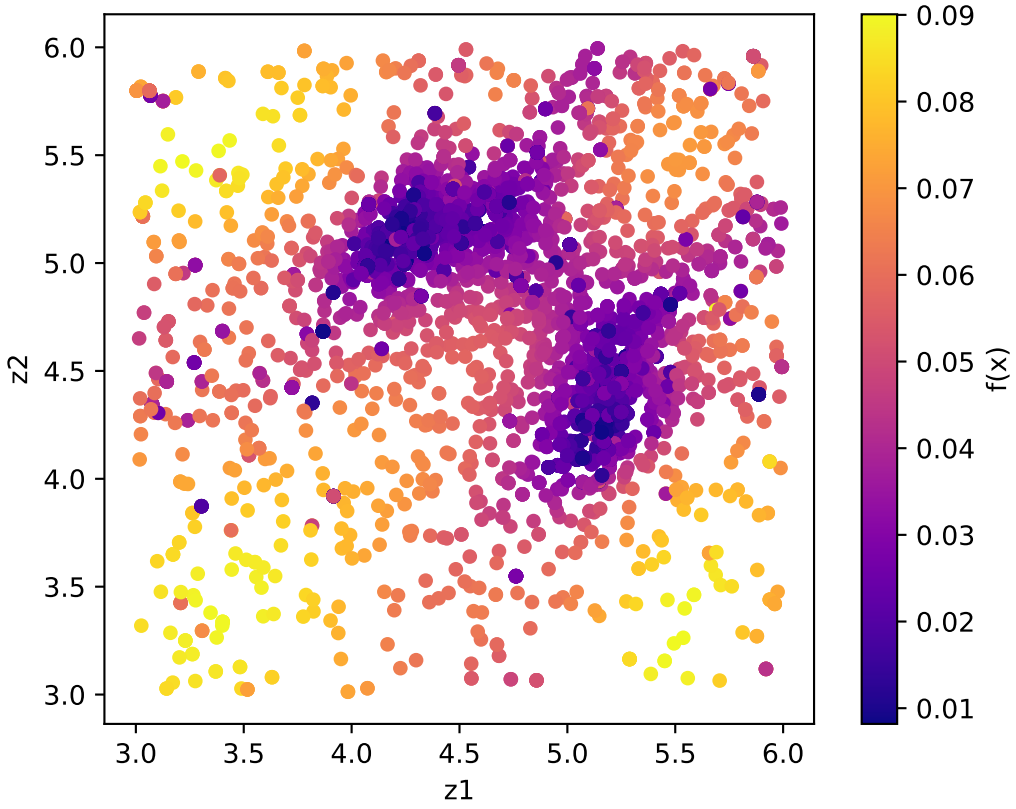


Fig. 3.6: Sampled parameters and `R-factor`. The horizontal axes is x1 (`value_01`) and the vertical axes is x2 (`value_02`).

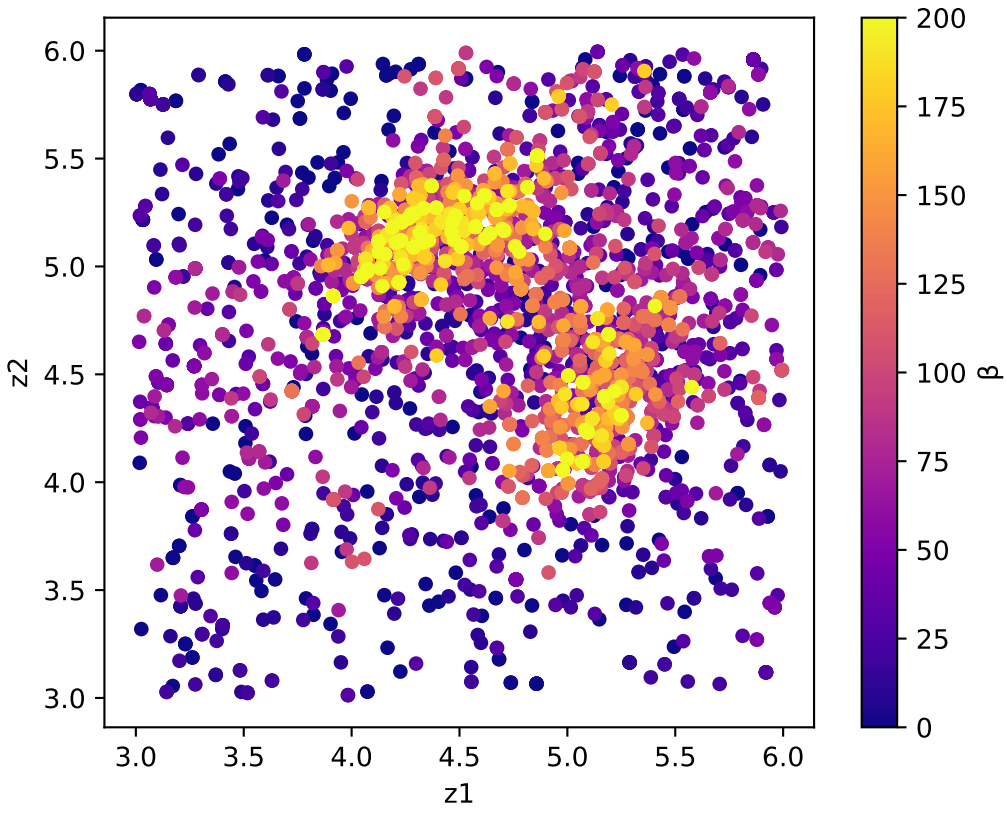


Fig. 3.7: Sampled parameters and β. The horizontal axes is x1 (`value_01`) and the vertical axes is x2 (`value_02`).

- `ref.txt`

  A file containing the answers you want to seek in this tutorial.

- `simple2.py`

  Another version of source file of the main program. The parameters are embedded in the program as a dict.

- `prepare.sh` , `do.sh`

  Script prepared for doing all calculation of this tutorial

The following sections describe these files and then show the actual calculation results.

3.7.2 Description of main program

`simple.py` is a simple program for the analyses using odatse-STR module. The entire source file is shown as follows:

```
import numpy as np

import odatse
import odatse.algorithm.min_search
from odatse.extra.STR import Solver

info = odatse.Info.from_file("input.toml")

odatse.mpi.setup()
solver = Solver(info)
runner = odatse.Runner(solver, info)
alg = odatse.algorithm.min_search.Algorithm(info, runner)
alg.main()
```

At the beginning of the program, the required modules are imported as listed below.

- `odatse` for the main module of ODAT-SE.
- `odatse.algorithm.min_search` for the module of the inverse problem algorithm used in this tutorial.
- `odatse.extra.STR` for the direct problem solver module.

Next, the instances of the classes are created.

- `odatse.Info` class

  This class is for storing the parameters. It is created by calling a class method `from_file` with a path to TOML file as an argument.

Before creating an instance of the Solver class, `odatse.mpi.setup()` is called to set up MPI. This call is mandatory in ODAT-SE v4, even when MPI is not used. (`odatse.initialize()`, which is used by the `odatse-STR` command, performs this setup internally together with the command-line argument processing.)

- `odatse.extra.STR.Solver` class

  This class is for the direct problem solver of the odatse-STR module. It is created by passing an instance of Info class.

- `odatse.Runner` class

  This class is for connecting the direct problem solver and the inverse problem algorithm. It is created by passing an instance of Solver class and an instance of Info class.

- `odatse.algorithm.min_search.Algorithm` class

This class is for the inverse problem algorithm. In this tutorial, we use `min_search` module that implements the optimization by Nelder-Mead method. It is created by passing an instance of Runner class.

After creating the instances of Solver, Runner, and Algorithm in this order, we invoke `main()` method of the Algorithm class to start analyses.

In the above program, the input parameters are read from a file in TOML format. It is also possible to take the parameters in the form of dictionary. `simple2.py` is anther version of the main program in which the parameters are embedded in the program. The entire source file is shown below:

```
import numpy as np

import odatse
import odatse.algorithm.min_search
from odatse.extra.STR import Solver

params = {
    "base": {
        "dimension": 3,
        "output_dir": "output",
    },
    "solver": {
        "run_scheme": "subprocess",
        "generate_rocking_curve": True,
        "config": {
            "cal_number": [1],
        },
        "param": {
            "string_list": ["value_01", "value_02", "value_03"],
        },
        "reference": {
            "path": "experiment.txt",
            "exp_number": [1],
        },
        "post": {
            "normalization": "TOTAL",
        },
    },
    "algorithm": {
        "label_list": ["z1", "z2", "z3"],
        "param": {
            "min_list": [ 0.0, 0.0, 0.0 ],
            "max_list": [ 10.0, 10.0, 10.0 ],
            "initial_list": [ 5.25, 4.25, 3.50],
        },
    },
}

info = odatse.Info(params)

odatse.mpi.setup()
solver = Solver(info)
runner = odatse.Runner(solver, info)
alg = odatse.algorithm.min_search.Algorithm(info, runner)
alg.main()
```

An instance of Info class is created by passing a set of parameters in a dict form. It is also possible to generate the parameters within the program before passing to the class.

3.7.3 Input files

The input file `input.toml` for the main program is the same as that used in the previous tutorial for Nelder-Mead method. Except, `algorithm.name` parameter for specifying the algorithm type should be ignored.

`template.txt` and `experiment.txt` are the same as those in the previous tutorials.

3.7.4 Calculation execution

First, move to the folder where the sample files are located. (We assume that you are directly under the directory where you downloaded this software.)

```
$ cd sample/single_beam/user_program
```

Copy `bulk.exe` and `surf.exe`.

```
$ cp ../../sim-trhepd-rheed/src/bulk.exe .
$ cp ../../sim-trhepd-rheed/src/surf.exe .
```

Run `bulk.exe` to produce `bulkP.b`.

```
$ ./bulk.exe
```

Then, run the main program. The computation time will take only a few seconds on a normal PC.

```
$ python3 simple.py | tee log.txt
```

The standard output will look as follows.

```
name            : minsearch
label_list      : ['z1', 'z2', 'z3']
param.min_list  : [0.0, 0.0, 0.0]
param.max_list  : [10.0, 10.0, 10.0]
param.initial_list: [5.25, 4.25, 3.5]
eval: x=[5.25 4.25 3.5 ], fun=0.015199252435883206
eval: x=[5.22916667 4.3125     3.64583333], fun=0.013702918645281299
eval: x=[5.22569444 4.40625    3.54513889], fun=0.01263527811899235
eval: x=[5.17997685 4.34895833 3.5943287 ], fun=0.006001659635528168
eval: x=[5.17997685 4.34895833 3.5943287 ], fun=0.006001659635528168
eval: x=[5.22066294 4.33260995 3.60408629], fun=0.005145496928704404
eval: x=[5.22066294 4.33260995 3.60408629], fun=0.005145496928704404
eval: x=[5.2185245  4.32627234 3.56743818], fun=0.0032531465329236025
eval: x=[5.19953437 4.34549482 3.5863457 ], fun=0.0027579225484420356
eval: x=[5.21549776 4.35100199 3.57476018], fun=0.002464573036316852
...
```

`x=` are the candidate parameters at each step, and `fun=` is the R-factor value at that point. The results at each step are also written in the folder `output/LogXXXX_YYYY` (where XXXX and YYYY are the step counts). The final estimated parameters will be written to `output/res.dat`. In the current case, the following result will be obtained:

```
z1 = 5.230524973874179
z2 = 4.370622919269477
z3 = 3.5961444501081647
```

You can see that we will get the same values as the correct answer data in `ref.txt`.

Note that `do.sh` is available as a script for batch calculation.

CHAPTER

FOUR

INPUT AND OUTPUT

odatse-STR module is a `Solver` package that uses sim-trhepd-rheed to calculate the diffraction rocking curve from the atomic position x and returns the deviation from the experimental rocking curve as $f(x)$.

> **Note**
>
> sim-trhepd-rheed is a serial program. The solver-level parallelization of ODAT-SE v4 (`--nsolve` greater than 1) is not supported, and specifying it results in an error. Use the algorithm-level parallelization (`--nalg`, which takes all processes by default) for MPI runs.

In this section, the input parameters, the input data, and the output data are explained. The input parameters are taken from the `solver` entry of the `Info` class. The parameters are specified in `[solver]` section when they are given from a TOML file. If the parameters are given in the dictionay format, they should be prepared as a nested dict under the `solver` key. In the following, the parameter items are described in the TOML format.

The input data consist of target reference data, and templates of the input file for sim-trhepd-rheed. The output data are the output files and log files generated by `surf.exe` of sim-trhepd-rheed. Their contents will be shown in this section.

4.1 Input parameters

Input parameters can be specified in the subsections `config`, `post`, `param`, `reference` in `solver` section.

4.1.1 [solver] section

- `generate_rocking_curve`

  Format: boolean (default: false)

  Description: Whether to generate `RockingCurve_calculated.txt`. If `true`, `RockingCurve_calculated.txt` will be generated in the working directory `Log%%%_###`. Note that when `remove_work_dir` (in `[post]` subsection) is `true`, `Log%%%_###` will be removed with the files in it.

4.1.2 [solver.config] subsection

- `surface_exec_file`

  Format: string (default: "surf.exe")

  Description: Path to `sim-trhepd-rheed` surface reflection solver `surf.exe`.

- `surface_input_file`

  Format: string (default: "surf.txt")

  Description: Input file for surface structure.

- `bulk_output_file`

  Format: string (default: "bulkP.b")

  Description: Output file for bulk structure.

- `surface_output_file`

  Format: string (default: "surf-bulkP.s")

  Description: Output file for surface structure.

- `calculated_first_line`

  Format: integer (default: 5)

  Description: In the output file, the first line to be read as D(x). The last line is automatically calculated from the number of the reference data.

- `calculated_info_line`

  Format: integer (default: 2)

  Description: In the output file, the line contains the information of the calculated data – the number of glancing angles (second column) and the number of beams (third column).

- `cal_number`

  Format: List of integers

  Description: The indices of columns in a file specified by `surface_output_file` that are to be used as data. Multiple columns can be set. The number of indices must be equal to that of `exp_number` (in [`reference`] section).

4.1.3 [`solver.post`] subsection

This subsection is used to the postprocess – to specify how to calculate the objective function, that is, the deviation between the experimental and computational data, and to draw the rocking curve.

- `Rfactor_type`

  Format: string ("A", "A2", or "B", default: "A")

  Description: This parameter specifies how to calculate the R-factor to be minimized.

  Let n be the number of dataset, m be the number of glancing angles, $u^{(n)} = (u_1^{(n)}, u_2^{(n)}, \ldots, u_m^{(n)})$ be the experimental data, and $v^{(n)} = (v_1^{(n)}, v_2^{(n)}, \ldots, v_m^{(n)})$ be the calculated data. With the weights of the beams, $w^{(j)}$, R-factors are defined as follows:

  - "A" type:

$$R = \sqrt{\sum_j^n w^{(j)} \sum_i^m \left(u_i^{(j)} - v_i^{(j)}\right)^2}$$

  - "A2" type:

$$R^2 = \sum_j^n w^{(j)} \sum_i^m \left(u_i^{(j)} - v_i^{(j)}\right)^2$$

- "B" type:

$$R = \frac{\sum_i^m \left(u_i^{(1)} - v_i^{(1)}\right)^2}{\sum_i^m \left(u_i^{(1)}\right)^2 + \sum_i^m \left(v_i^{(1)}\right)^2}$$

"B" type is available only for the single dataset ($n = 1$).

- `normalization`

  Format: string ("TOTAL" or "MANY_BEAM", mandatory)

  Description: This parameter specifies how to normalize the experimental and computed data vectors.

  - "TOTAL"
    - To normalize the data as the summation is 1.
    - The number of dataset should be one (the length of `cal_number` should be one).
  - "MANY_BEAM"
    - To normalize with weights as specified by `weight_type`.

  NOTE: "MAX" is no longer available

- `weight_type`

  Format: string or `None`. "calc" or "manual" (default: `None`)

  Description: The weights of the datasets for the "MANY_BEAM" normalization.

  - "calc"

$$w^{(n)} = \left(\frac{\sum_i^m v_i^{(n)}}{\sum_j^n \sum_i^m v_i^{(j)}} \right)^2$$

  - "manual"

    $w^{(n)}$ is specified by `spot_weight`.

- `spot_weight`

  Format: list of float (mandatory when `weight_type` is "manual")

  Description: The weights of the beams in the calculation of R-factor. The weights are automatically normalized as the sum be 1. For example, [3,2,1] means $w^{(1)} = 1/2, w^{(2)} = 1/3, w^{(3)} = 1/6$.

- `omega`

  Format: float (default: 0.5)

  Description: This parameter specifies the half-width of convolution.

- `remove_work_dir`

  Format: boolean (default: false)

  Description: If `True`, working directories `Log%%%_###` will be removed after reading R-factor.

4.1.4 [solver.param] subsection

- string_list

  Format: list of string. The length should match the value of dimension (default: ["value_01", "value_02"]).

  Description: List of placeholders to be used in the reference template file to create the input file for the solver. These strings will be replaced with the values of the parameters being searched for.

4.1.5 [solver.reference] subsection

- path

  Format: string (default: experiment.txt)

  Description: Path to the reference data file.

- reference_first_line

  Format: integer

  Description: In the reference data file, the first line to be read as experimental data. The default value is 1, that is, the first line of the file.

- reference_last_line

  Format: integer

  Description: In the reference data file, the last line to be read as experimental data. If omitted, all lines from the first line to the end of the file will be read.

- exp_number

  Format: List of integers

  Description: In the reference data file, the column numbers to be read. Multiple columns can be specified (many-beam condition).

4.2 Reference files

4.2.1 Input template file

The input template file template.txt is a template for creating an input file for surf.exe. The parameters to be varied in odatse-STR (such as the atomic coordinates you want to find) should be replaced with the appropriate string, such as value_*. The strings to be used are specified by string_list in the [solver] - [param] section of the input file for the solver. An example template is shown below.

```
2                                   ,NELMS,  -------- Ge(001)-c4x2
32,1.0,0.1                          ,Ge Z,da1,sap
0.6,0.6,0.6                         ,BH(I),BK(I),BZ(I)
32,1.0,0.1                          ,Ge Z,da1,sap
0.4,0.4,0.4                         ,BH(I),BK(I),BZ(I)
9,4,0,0,2, 2.0,-0.5,0.5              ,NSGS,msa,msb,nsa,nsb,dthick,DXS,DYS
8                                   ,NATM
1, 1.0, 1.34502591  1       value_01   ,IELM(I),ocr(I),X(I),Y(I),Z(I)
1, 1.0, 0.752457792 1       value_02
2, 1.0, 1.480003343 1.465005851     value_03
2, 1.0, 2   1.497500418     2.281675
2, 1.0, 1   1.5     1.991675
2, 1.0, 0   1       0.847225
```

(continues on next page)

(continued from previous page)

```
2, 1.0, 2   1       0.807225
2, 1.0, 1.009998328 1       0.597225
1,1                                   ,(WDOM,I=1,NDOM)
```

In this case, `value_01`, `value_02`, and `value_03` are the parameters to be varied in `odatse-STR`.

4.2.2 Target file

This file (`experiment.txt`) contains the data to be targeted. The first column contains the angle, and the second and following columns contain the calculated value of the reflection intensity multiplied by the weight. An example of the file is shown below.

```
3.00000e-01 8.17149e-03 1.03057e-05 8.88164e-15 ...
4.00000e-01 1.13871e-02 4.01611e-05 2.23952e-13 ...
5.00000e-01 1.44044e-02 1.29668e-04 4.53633e-12 ...
6.00000e-01 1.68659e-02 3.49471e-04 7.38656e-11 ...
7.00000e-01 1.85375e-02 7.93037e-04 9.67719e-10 ...
8.00000e-01 1.93113e-02 1.52987e-03 1.02117e-08 ...
9.00000e-01 1.92590e-02 2.53448e-03 8.69136e-08 ...
1.00000e+00 1.86780e-02 3.64176e-03 5.97661e-07 ...
1.10000e+00 1.80255e-02 4.57932e-03 3.32760e-06 ...
1.20000e+00 1.77339e-02 5.07634e-03 1.50410e-05 ...
1.30000e+00 1.80264e-02 4.99008e-03 5.53791e-05 ...
...
```

4.3 Output files

For `sim-trhepd-rheed`, the files output by `surf.exe` will be output in the `Log%%%%%_#####` folder under the folder with the rank number. `%%%%%` means an index of iteration in `Algorithm` (e.g., steps in Monte Carlo), and `#####` means an index of group (e.g., replica index in Monte Carlo). In large calculation, the number of these folders becomes too large to be written in the storage of the system. For such a case, let `solver.post.remove_work_dir` parameter be `true` in order to remove these folders. This section describes the own files that are output by this solver.

4.3.1 `stdout`

It contains the standard output of `surf.exe`. Usually it is empty.

4.3.2 `RockingCurve_calculated.txt`

This file will be generated in the `Log%%%%%_#####` folder if `generate_rocking_curve` (in `[solver]` section) is set to "true".

At the beginning of the file, the lines beginning with `#` are headers. The header contains the values of the input variables, the objective function value `f(x)`, the parameters `Rfactor_type`, `normalization`, `weight_type`, `cal_number`, `spot_weight`, and what is marked in the data portion columns (e.g. `# #0 glancing_angle`).

The header is followed by the data. The first column shows the glancing angle, and the second and subsequent columns show the intensity of each data column. You can see which data columns are marked in the header. For example,

```
# #0 glancing_angle
# #1 cal_number=1
# #2 cal_number=2
# #3 cal_number=4
```

shows that the first column is the glancing angle, and the second, third, and fourth columns are the calculated data corresponding to the first, second, and fourth columns of the calculated data file, respectively.

Intencities in each column are normalized so that the sum of the intensity is 1. To calculate the objective function value (R-factor and R-factor squared), the data columns are weighted by `spot_weight` and normalized by `normalization`.

```
#value_01 =  0.00000 value_02 =  0.00000
#Rfactor_type = A
#normalization = MANY_BEAM
#weight_type = manual
#fx(x) = 0.03686180462340505
#cal_number = [1, 2, 4, 6, 8]
#spot_weight = [0.933 0.026 0.036 0.003 0.002]
#NOTICE : Intensities are NOT multiplied by spot_weight.
#The intensity I_(spot) for each spot is normalized as in the following equation.
#sum( I_(spot) ) = 1
#
# #0 glancing_angle
# #1 cal_number=1
# #2 cal_number=2
# #3 cal_number=4
# #4 cal_number=6
# #5 cal_number=8
0.30000 1.278160358686800e-02 1.378767858296659e-04 8.396046839668212e-14 1.
↪342648818357391e-30 6.697979700048016e-53
0.40000 1.778953628930054e-02 5.281839702773564e-04 2.108814173486245e-12 2.
↪467220122612354e-28 7.252675318478533e-50
0.50000 2.247181148723425e-02 1.671115124520428e-03 4.250758278908295e-11 3.
↪632860054842994e-26 6.291667506376419e-47
...
```

CHAPTER

FIVE

ACKNOWLEDGEMENTS

The development of odatse-STR was supported by “Project for advancement of software usability in materials science” (FY2020, 2021, 2024) of The Institute for Solid State Physics, The University of Tokyo.

For the implementation of the forward problem solver, we thank T. Hanada (Tohoku Univ.) and I. Mochizuki (KEK).

CHAPTER

SIX

CONTACT

- Bug Reports

  Please report all problems and bugs on the github Issues page.

  To resolve bugs early, follow these guidelines when reporting:

  1. Please specify the version of ODAT-SE and odatse-STR you are using.
  2. If there are problems for installation, please inform us about your operating system and the compiler.
  3. If a problem occurs during execution, enter the input file used for execution and its output.

  Thank you for your cooperation.

- Others

  If you have any questions about your research that are difficult to consult at Issues on GitHub, please send an e-mail to the following address:

  E-mail: `2dmat-dev__at__issp.u-tokyo.ac.jp` (replace _at_ by @)